UnionLabs SDR Platform — Beginner's Guide

Contents

UnionLabs SDR Platform — Beginner's Guide

A single, self-contained guide: what the platform is, how to run it, how to add your own algorithm, and how to drive the radio directly. No prior knowledge of the codebase needed.

> **PHY = the "modem".** It turns your data (bytes / a float32 vector) into radio waves and back: framing -> error-correction (FEC) -> modulation -> radio, and the reverse on receive. You bring an *algorithm*; the PHY moves its data over the air.
>
> **The promise.** You write the algorithm once. The same file runs with no radio, over a USRP software-defined radio, or over a LoRa module — you change a flag, not your code.

---

1. Quick start

Install first — Python 3.8 or newer, one command, no radio and no C++ build:

```
pip install -r requirements.txt
./run.sh selftest           # confirm the install works (every experiment x every radio-free PHY)
```

If `selftest` prints `all N checks passed`, everything below will work. If only the `usrp` checks fail, the compiled `pyphy` extension does not match your Python — build it with `drivers/usrp/bindings/build.sh`, or just use `--channel ideal` / `--channel lora`, which need nothing.

```
./run.sh                    # run an algorithm over the PHY (defaults: echo, no radio)
./run.sh list               # list the algorithms available
./run.sh --algo marl        # run a specific algorithm
./run.sh --help             # help + every option
./radio.sh rx               # raw receive on a USRP (B210 default; --device n210|x310)
./radio.sh tx               # raw transmit
```

Anything you don't specify takes a **default** (`algo=echo role=loopback channel=ideal steps=5`). `run.sh` prints the exact command it runs (the `>>` line) so you learn the underlying call.

2. Where things live

- **`experiments/`** — **everything you run**, one folder per experiment, each with an `app.py`. Your own code and the worked examples (FL, decentralized learning, MARL, CLIP semantic comm, STC-AirComp, jammer) all live here — one place to look.
- **`union/`** — the middleware you code against: `run_algo.py` + `phy_link.py` (the uniform API) and `driver.py` (the contract every PHY implements).
- **`drivers/`** — the physical layers, one folder each:
  - `usrp/build/sdr_system` — the C++ radio engine (the modem); `radio.sh` runs raw TX/RX.
  - `usrp/python/` — `sdr.py` / `run.py` / `configs/` (direct radio), `channel_sense.py` (RF utils).
  - `usrp/bindings/pyphy` — the DSP blocks as numpy functions.
  - `lora/` — the SX1276 LoRa PHY (firmware + the Python driver).
- **`docs/`** — every guide, reference and PDF, including this one.

Three ways to use the PHY, easiest first:

1. **Run an algorithm** with `run.sh` (§3).
2. **Add your own algorithm** (§4).
3. **Drive the radio directly** with `sdr.py` / `radio.sh` (§5), or compose the DSP blocks (§6).

---

3. Running an algorithm over the PHY

Your algorithm only says **what to transmit** and **what to receive** — no radio code. Drop it in `experiments/<name>/app.py` (copy `experiments/_template/`), then run it by name.

3.1 The three independent choices

Every run answers three separate questions. Keeping them apart is the single most useful thing to understand about this platform:

```
./run.sh  --algo fl        --channel lora        --role loopback
          └─ WHAT ─────┘   └─ WHICH PHY ──┘      └─ WHICH PART am I ─┘
```

1. **`--algo`** — what you are running. Any folder in `experiments/` (`./run.sh list`).
2. **`--channel`** — which physical layer carries the bytes. Your algorithm never knows.
3. **`--role`** — whether this process is the *whole network* or *one node of it*.

A fourth, `--<phy>-backend`, says how the PHY is attached. **Every default backend needs no hardware**, so you develop the whole experiment on a laptop and change one flag to go on air.

3.2 Choosing a PHY (`--channel`)

`--channel`	What it is	Needs hardware?
`ideal` *(default)*	lossless, in-process	no
`usrp`	the real C++ modem (OFDM/single-carrier, LDPC/turbo FEC, sync, CFO)	no, with `--usrp-backend pyphy`

`--channel`	What it is	Needs hardware?
`lora`	the SX1276 LoRa PHY (255-byte MTU, fragmentation + ARQ)	no, with `--lora-backend sim`

```
./run.sh --algo fl                                  # ideal — is my logic right?
./run.sh --algo fl --channel usrp --snr-db 6        # how noisy can the link get?
./run.sh --algo fl --channel lora --lora-sf 9       # what does this cost on LoRa?
```

Older spellings still work: `--channel sim` = `ideal`, `--channel pyphy` = `usrp`.

3.3 Choosing a role (`--role`)

Group roles build the whole network in one process — this is how you develop:

`--role`	Shape	Extra flags
`loopback` *(default)*	two ends, one round-trip each step	—
`chain`	initiator -> relay(s) -> responder, every hop over the PHY	`--relays 1`
`gossip`	N peers exchanging over a graph, no server	`--agents 6 --topology`
`multi`	N agents contending for one access point (collisions are real)	`--agents 4`
`aircomp`	N sensors transmit at once; the air sums them	`--agents 8`

Node roles run ONE node in this process — this is how you deploy, one terminal or one computer per node:

`--role`	This process is	Extra flags
`tx`	the initiator (transmits, then listens)	`--radio`, peer host
`rx`	the responder (listens, then answers)	`--radio`
`relay`	a middle node: receives from upstream, re-transmits downstream	`--down-host`, `--down-port`
`peer`	one node of a decentralized network — both TX and RX, at different steps	`--node K --agents N`

```
# develop: the whole 6-peer network in one process
./run.sh --algo dl --role gossip --agents 6 --topology ring

# deploy: one terminal per node (--node K implies --role peer)
./run.sh --algo dl --node 0 --agents 3
./run.sh --algo dl --node 1 --agents 3
./run.sh --algo dl --node 2 --agents 3
```

> **An algorithm can name its own roles.** `fl` calls them `client/server`, `dl` calls them `initiator/responder/peer`. `./run.sh list` prints each algorithm's roles, and you type the algorithm's own name: `./run.sh --algo fl --role server`. `tx/rx` always work too.

3.4 Every `run.sh` option

Core

Option	Default	What it does
`--algo <name>`	`echo`	which folder under `experiments/` to run
`--channel ideal|usrp|lora`	`ideal`	which PHY carries the payloads
`--role <role>`	`loopback`	see §3.3; or any role the algorithm declares
`--steps <n>`	`5`	how many rounds/round-trips to run
`--agents <n>`	`4`	number of nodes for `gossip` / `multi` / `aircomp`

Option	Default	What it does
`--node <k>`	—	run ONE node of a decentralized network (implies `--role peer`)
`--relays <n>`	`1`	relay nodes in the middle of a `chain`
`--topology <t>`	`ring`	`gossip` graph: `ring`, `full`, or an edge list `0-1,1-2,2-0`
`--snr-db <dB>`	`8`	link signal-to-noise ratio (both PHYs)

Shared radio options (any PHY)

Option	Default	What it does
`--freq <MHz>`	`915`	centre frequency. Outside 902–928 MHz you get a US-band warning
`--arq <scheme>`	`stop-and-wait`	retransmission policy (only one implemented so far)
`--max-attempts <n>`	per PHY	give up on a chunk after this many un-ACKed sends (USRP 50, LoRa 8)

USRP options (`--channel usrp`) — we assemble this PHY, so every part is yours to choose

Option	Default	What it does
`--usrp-backend pyphy|radio`	`pyphy`	in-process modem, or real radios (needs two hosts)
`--modulation <NAME>`	`QPSK`	BPSK, QPSK, 8-PSK, 16-QAM, DBPSK, DQPSK (`--scheme` is the same flag)
`--fec conv|ldpc|turbo|""`	`turbo`	error-correcting code; "" turns coding off
`--samp-rate` / `--symbol-rate`	`2e6` / `1e6`	sample and symbol rate in Hz
`--tx-gain` / `--rx-gain`	`70` / `30`	transmit and receive gain in dB
`--ack-transport tcp|rf`	`tcp`	where the ACK travels: a socket, or a second RF path
`--ack-timeout <ms>`	`3000`	how long the sender waits for an ACK
`--radio serial=…|addr=…`	—	which USRP this process owns (B210 by serial, N210/X310 by address)
`--tx-args` / `--rx-args`	""	as above, when one node has two radios

LoRa options (`--channel lora`) — the chip embeds modulation and CRC, so these are its knobs

Option	Default	What it does
`--lora-backend sim|serial|spi`	`sim`	no hardware / Arduino on USB / Pi with the radio on SPI
`--lora-sf <7..12>`	`9`	spreading factor: higher reaches further, costs exponentially more airtime
`--lora-cr <5..8>`	`5`	coding rate denominator (4/5 … 4/8)
`--lora-bw 125000|250000|500000`	`125000`	bandwidth in Hz; doubling it halves airtime and costs ∼3 dB sensitivity
`--lora-power <dBm>`	`14`	transmit power
`--lora-port <dev>`	—	serial backend: e.g. `/dev/ttyUSB0`
`--lora-verbose`	off	print fragments, retransmissions and airtime for every message

Multi-process options (node roles)

Option	Default	What it does
`--peers <h1,h2,…>`	all localhost	one host per node id, when nodes are on different computers
`--peer-port <n>`	`5800`	base TCP port; node k listens on `peer-port + k`
`--peer-link tcp|wireless|lora`	follows `--channel`	how decentralized peers exchange
`--ack-host` / `--net-host`	`127.0.0.1`	the peer host's IP (USRP `tx`/`rx` roles)

Passing a knob for a PHY you did not select prints a NOTE rather than being silently ignored — `--lora-sf` with `--channel usrp` tells you so.

3.5 Reading the output

```
[run_algo] loaded experiments/fl via make(role) binding
[run_algo] role 'client' -> PHY end 'tx'
  step 0: req_ber=0.0000 rep_ber=0.0000 delivered=True
[run_algo] loopback done: 12/12 round-trips delivered over channel=ideal
[run_algo] lora PHY: {'msgs': 4, 'frags': 416, 'retx': 0, 'lost': 0,
                      'airtime_s': 165.09, 'sf': 7, 'arq': 'stop-and-wait'}
```

- **`delivered`** — the round-trip completed and the CRC was clean.
- **`req_ber` / `rep_ber`** — bit error rate each way (the USRP modem reports real errors; LoRa reports 0 because the chip only surfaces CRC-valid packets, so a bad frame is a *loss*).
- **The PHY line** — what the run actually cost. For LoRa this is the headline: `frags` is how many 247-byte pieces your payload needed, `retx` the retransmissions, and `airtime_s` the real Semtech airtime. A 25 kB model over two rounds costs **165 s at SF7 and 3727 s at SF12** — the same experiment, 22× the time on air.

To write your own `app.py`, see §4.

---

4. Add your own algorithm

This has a guide of its own, so it stays in one place rather than drifting between two:

> **->** **`HOW_TO_ADD_ALGORITHM.md`** (repo root, also as `docs/HOW_TO_ADD_ALGORITHM.pdf`)

It covers, in order:

1. **Where to put it** — `experiments/<name>/app.py`, and `--algo <name>` matches the folder.
2. **One file or many** — many: `app.py` is only the bridge, and your own modules, sub-packages and data sit beside it.
3. **What `app.py` must provide** — `make(role)` returning an object with `transmit()` / `receive(msg)`, plus optional `spec` and `on_result(ack)`.
4. **Two ways to write it** — inline (copy `experiments/_template/`), or a ~10-line binding onto your existing, untouched code (copy `experiments/plain_echo/`).
5. **Roles** — the four node types (`tx`, `rx`, `relay`, `peer`), naming your own with `ROLES`, and learning which node you are with `make(role, index, total)`.
6. **Running it** — the same file over every PHY, and as a multi-node network.

The shortest possible version:

```python
# experiments/my_algo/app.py
import numpy as np

class MyAlgo:
    spec = ("float32", (8,))
    def __init__(self, role): self.role = role
    def transmit(self):      return np.ones(8, np.float32)   # what to transmit (None = done)
    def receive(self, msg):  print("got", msg)               # what to receive

def make(role): return MyAlgo(role)
```

```
./run.sh --algo my_algo
```

5. Driving the radio directly (`sdr.py` / `radio.sh`)

When you want raw control of the modem (no algorithm layer), use the roles.

Shortcut: `./radio.sh tx` or `./radio.sh rx` runs a raw TX/RX on a USRP with the right per-device defaults — **B210 is the default**; add `--device n210` or `--device x310`. E.g. `./radio.sh rx --device n210 --freq 915e6`. Add `--dry-run` to just print the command.

From Python (`drivers/usrp/python/sdr.py`)

```
from sdr import tx, rx, both, source_arq, sink_arq, run_pair, options
options()                                          # print every option + default
tx(tx_args="serial=30CD424", scheme="QPSK", tx_gain=78, fec=True).run()   # transmit only
rx(rx_args="serial=30CD3F7", scheme="QPSK", fec=True).run()               # receive only
# reliable stop-and-wait ARQ across two radios (RX first):
run_pair(sink_arq(rx_args="serial=30CD3F7", scheme="QPSK", fec=True),
         source_arq(tx_args="serial=30CD424", scheme="QPSK", fec=True))
print(tx(scheme="QPSK").command())                 # just print the command (don't run)
```

The roles

Role	Meaning
`tx` / `rx`	transmit only / receive only (one radio each)
`both`	transmit **and** receive in one process (single-box loopback)
`source_arq` / `sink_arq`	the two ends of reliable stop-and-wait ARQ (with ACK)
`sense`	listen and report channel power (used by `channel_sense.py`)

The options you'll actually use (curated — the full list is `PARAMETERS.md`, or `sdr_system --help`)

Option	Example	Meaning
`--role`	`source_arq`	what this process does (table above)
`--scheme`	`QPSK`	modulation: `BPSK QPSK 8-PSK 16/64-QAM`, differential `DBPSK`
`--waveform`	`sc`	`sc` (single-carrier) or `ofdm` (many subcarriers, tracks freque
`--fec`	`true`	turn error-correction on
`--fec-type`	`turbo`	`conv` (fast), `ldpc`, or `turbo` (strongest)
`--fec_soft`	`true`	soft-decision decoding (better; pair with ldpc/turbo)
`--tx-args` / `--rx-args`	`serial=30CD424` / `addr=192.168.20.2`	pick the radio (B210 serial / N210 addr)
`--tx-freq` / `--rx-freq`	`915e6`	carrier frequency (Hz)
`--tx-rate` / `--rx-rate`	`2e6`	sample rate (Hz); N210 needs `2e6`
`--symbol-rate`	`1e6`	symbol rate; N210 pairs `2e6/1e6`
`--tx-gain` / `--rx-gain`	`78` / `30`	amplifier gain (dB)
`--tx-subdev` / `--rx-subdev`	`A:A` / `A:0`	radio front-end (B210 `A:A`, N210 `A:0`)
`--ack-transport`	`tcp`	send the ARQ ACK over `tcp` (so an RX-only radio never tra
`--ack-host` / `--ack-port`	`192.168.1.50` / `5599`	where the ACK goes
`--bytes-length`	`125`	payload size per frame (bytes)
`--max-attempts`	`1`	retransmit limit; `1` = single-shot, `0`/high = keep trying
`--viz` / `--viz-dir`	`true` / `phy_outputs`	capture TX/RX signals + auto-plot to `phy_outputs/<scheme>`
`--config <file>`	`phy.cfg`	load all options from a file (CLI still overrides)

Two rules that save hours: **start the receiver before the transmitter**, and on free-running radios use **`DQPSK`** (single-carrier) or **`OFDM`** — plain coherent `QPSK` delivers poorly without a shared clock.

From a JSON file (`drivers/usrp/python/run.py`)

```
python3 drivers/usrp/python/run.py --list                                  # ready-made configs
python3 drivers/usrp/python/run.py configs/qpsk_arq_pair.json --dry-run    # print, don't run
```

---

6. The building blocks (`pyphy`)

Every DSP stage is also a plain Python function, so you can compose the chain yourself (GNU-Radio style). Build it with `drivers/usrp/bindings/build.sh`; import with `PYTHONPATH=drivers/usrp/bindings python3` (add `arch -x86_64` on macOS).

Group	Functions
Framing	`frame`, `unframe`
FEC	`fec_encode`, `fec_decode`, `fec_decode_soft`, `fec_encoded_len` (`conv|ldpc|turbo`, `k`)
Modulation	`modulate`, `demodulate`, `soft_llr`
Single-carrier	`rrc_tx`, `rrc_rx` (pulse shape / matched filter)
Sync	`preamble`, `acq`, `cfo_correct`, `phase_correct`
OFDM	`ofdm_mod`, `ofdm_demod`, `ofdm_data_per_sym`
Radio (UHD build)	`Radio(role, args, freq, rate, symbol_rate, gain, subdev, ant)` · `.transmit()` · `.capture()`

```
import numpy as np, pyphy
bits = (np.random.rand(2000) > 0.5).astype(np.uint8)
syms = pyphy.modulate(bits, "QPSK")            # bits -> symbols
syms = syms * 0.6                              # <- your own op between stages
wave = pyphy.rrc_tx(syms, sps=2, beta=0.25)    # pulse-shape to a waveform
# ... channel/radio ...
back = pyphy.demodulate(pyphy.rrc_rx(wave, sps=2, beta=0.25)[::2], "QPSK")
```

Worked flowgraph: `drivers/usrp/python/phy_flow_example.py`.

---

7. Function cheat-sheet (the Python API)

Uniform API — `union/phy_link.py`

- `adapt(obj, role)` — wrap ANY object with `transmit()`/`receive()` into the internal app.
- `PayloadSpec(dtype, shape)` — declare an output's type + shape.
- `Codec.pack(arr)` / `Codec.unpack(buf)` — numpy array <-> self-describing bytes.
- `IdealChannel` / `PyphyChannel(scheme, fec, snr_db)` — radio-free channels.
- `run_loopback(tx, rx, channel, steps)` — radio-free round-trip driver.
- `RadioRoundTrip(role, …).step(app)` — the two-host radio round-trip.

Radio wrapper — `drivers/usrp/python/sdr.py`

- `SDR(**opts)` — one invocation; `.command()` (print), `.run()` (blocking), `.popen()` (background), `.set(**opts)`.
- helpers `tx / rx / both / source_arq / sink_arq / run_pair / options`.

Spectrum — `drivers/usrp/python/channel_sense.py`, `freq_scan.py`

- `sense_channel(...)` — measure power at one frequency.
- `python3 drivers/usrp/python/freq_scan.py --start 902 --stop 928 --rx-args addr=192.168.20.2` — find a quiet carrier.

---

8. Troubleshooting (beginner gotchas)

Symptom	Fix
`pyphy` import / `Python.h` / arch error (macOS)	run via `./run.sh` (it sets `PYTHONPATH` + `arch -x86_64`), or build with `drivers/us`
`Exec format error` on `sdr_system`	rebuild on that machine: `cd phy && cmake -S . -B build && cmake --build b`
receiver gets nothing / no ACK	start the **receiver first**; keep both radios on; use `DQPSK` or `OFDM`
startup "rate check" fails	N210 needs `--rx-rate 2e6 --symbol-rate 1e6`; set `--tx-rate = --rx-rate`
a stuck receiver won't stop	press Ctrl-C again; last resort `Ctrl-\` or `kill -9`
coherent `QPSK` ~17% delivered	free-running clocks — use `DQPSK` (single-carrier) or `OFDM`

Beyond this guide

For deeper reference (in the repo root / `experiments/`): **`PARAMETERS.md`** (every option — or run `sdr_system --help`), **`SYSTEM_REFERENCE.pdf`** (the engine math + every algorithm), **`COMMANDS.md`** and **`USRP_CARRIER_MODULATION.txt`** (ready-to-run command recipes per scheme / per device), **`HARDWARE.md`** (hardware setup), and **`experiments/EXPERIMENT_GUIDE.pdf`** (running the example apps).